EX27 · CLASS_17_VECTORS · 02 / GUÍA DE REFERENCIA PYTHON

NumPy para productos vectoriales

Funciones, entradas, salidas y ejemplos pequeños; sin resolver el ejercicio completo.

Ruta de aprendizaje
01 Teoría matemática → 02 Referencia Python → 03 Ejercicio → 04 Pseudocódigo → Programación.

Requisitos previos: EX25 y EX26, arrays NumPy, norma 2 y producto escalar.

1. Antes de empezar

Mantendremos un único archivo Python para EX27. Importa NumPy como en EX25 y EX26:

```
import numpy as np
```

Un vector 3D puede definirse a partir de una lista. Escribe las componentes en el orden x, y, z.

```
v = np.array([1.0, 2.0, 3.0])
print(v.shape)       # (3,)
print(v[0])          # 1.0
```

2. np.cross(a, b)

Elemento	Descripción
Entrada a	Primer vector NumPy 3D.
Entrada b	Segundo vector NumPy 3D.
Devuelve	Array con 3 componentes: vector a × b.
Orden	a × b no es b × a.

```
a = np.array([1.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.0])
c = np.cross(a, b)
print(c)             # [0. 0. 1.]
```

Observa que `np.cross` devuelve un vector, mientras que `np.dot` devuelve un escalar para dos vectores unidimensionales.

Advertencia sobre 2D
Para este curso introducimos coordenadas z = 0 en problemas planos. Así trabajamos siempre con arrays de tres componentes y evitamos diferencias de comportamiento entre versiones de NumPy.

3. np.dot(a, b)

Recibe dos arrays compatibles y devuelve el producto escalar cuando ambos son vectores 1D. Nos permite comprobar la perpendicularidad del vector calculado.

```
u = np.array([1.0, 0.0, 0.0])
v = np.array([0.0, 0.0, 1.0])
print(np.dot(u, v))  # 0.0
```

4. np.linalg.norm(v)

Devuelve la norma euclídea de un vector 1D (un número real no negativo). Para un vector resultado, indica su magnitud.

```
v = np.array([0.0, 3.0, 4.0])
magnitude = np.linalg.norm(v)
print(magnitude)     # 5.0
```

5. Acceso a componentes: shape y corchetes

El atributo `.shape` describe las dimensiones del array. Con `v[0]`, `v[1]`, `v[2]` accedemos respectivamente a x, y, z.

```
v = np.array([4.0, -2.0, 6.0])
print(v.shape)       # (3,)
print(v[2])          # 6.0
```

6. Cómo comprobar un resultado

Primero calcula el producto manualmente. Después verifica su perpendicularidad con `np.dot` y compara la magnitud con una cuenta manual. Puedes utilizar números cercanos a cero por efectos de redondeo.

```
x = np.array([1.0, 0.0, 0.0])
y = np.array([0.0, 1.0, 0.0])
z = np.cross(x, y)
print(np.dot(z, x))  # 0.0
print(np.dot(z, y))  # 0.0
```

7. Errores frecuentes

Situación	Cómo evitarla
Intercambiar argumentos	Respetar `np.cross(r, F)` para momentos.
Usar * entre arrays	* multiplica componente a componente, no calcula a × b.
Omitir la tercera componente	Escribir [x,y,0] en problemas planos.
Olvidar unidades	Momento en N·m; no en N ni J.

8. Mini resumen

```
np.array([x, y, z])   # vector 3D
np.cross(a, b)        # vector perpendicular
np.dot(a, b)          # producto escalar / dot product
np.linalg.norm(a)     # magnitud / magnitude
a.shape               # forma / shape
```